

cartpole_balance.py

```

1  """
2  Solution code for the problem "Cart-pole balance".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7  import numpy as np
8  from scipy.integrate import odeint
9  import matplotlib.pyplot as plt
10
11 from animations import animate_cartpole
12
13 # Constants
14 n = 4 # state dimension
15 m = 1 # control dimension
16 mp = 2.0 # pendulum mass
17 mc = 10.0 # cart mass
18 L = 1.0 # pendulum length
19 g = 9.81 # gravitational acceleration
20 dt = 0.1 # discretization time step
21 animate = False # whether or not to animate results
22
23
24 def cartpole(s: np.ndarray, u: np.ndarray) -> np.ndarray:
25     """Compute the cart-pole state derivative
26
27     Args:
28         s (np.ndarray): The cartpole state: [x, theta, x_dot, theta_dot], shape (n,)
29         u (np.ndarray): The cartpole control: [F_x], shape (m,)
30
31     Returns:
32         np.ndarray: The state derivative, shape (n,)
33     """
34     x, theta, dx, dtheta = s
35     sintheta, costheta = np.sin(theta), np.cos(theta)
36     h = mc + mp * (sintheta**2)
37     ds = np.array(
38         [
39             dx,
40             dtheta,
41             (mp * sintheta * (L * (dtheta**2) + g * costheta) + u[0]) / h,
42             -((mc + mp) * g * sintheta + mp * L * (dtheta**2) * sintheta * costheta + u[0] * costheta)
43             / (h * L),
44         ]
45     )
46     return ds
47
48
49 def reference(t: float) -> np.ndarray:
50     """Compute the reference state (s_bar) at time t
51
52     Args:
53         t (float): Evaluation time
54
55     Returns:
56         np.ndarray: Reference state, shape (n,)
57     """
58     a = 10.0 # Amplitude
59     T = 10.0 # Period
60
61     # PART (d) #####
62     return np.array([
63         a * np.sin(2 * np.pi * t / T),
64         np.pi,
65         a * (2 * np.pi / T) * np.cos(2 * np.pi * t / T),
66         0.0,
67     ])
68     # END PART (d) #####
69
70
71 def ricatti_recursion(

```

```

72 A: np.ndarray, B: np.ndarray, Q: np.ndarray, R: np.ndarray
73 ) -> np.ndarray:
74 """Compute the gain matrix K through Ricatti recursion
75
76 Args:
77     A (np.ndarray): Dynamics matrix, shape (n, n)
78     B (np.ndarray): Controls matrix, shape (n, m)
79     Q (np.ndarray): State cost matrix, shape (n, n)
80     R (np.ndarray): Control cost matrix, shape (m, m)
81
82 Returns:
83     np.ndarray: Gain matrix K, shape (m, n)
84 """
85 eps = 1e-4 # Riccati recursion convergence tolerance
86 max_iters = 1000 # Riccati recursion maximum number of iterations
87 P_prev = np.zeros((n, n)) # initialization
88 converged = False
89 for i in range(max_iters):
90     # PART (b) #####
91     K = -np.linalg.inv(R + B.T @ P_prev @ B) @ B.T @ P_prev @ A
92     P = Q + A.T @ P_prev @ (A + B @ K)
93     if np.max(np.abs(P - P_prev)) < eps:
94         converged = True
95         break
96     P_prev = P
97     # END PART (b) #####
98 if not converged:
99     raise RuntimeError("Ricatti recursion did not converge!")
100 print("K:", K)
101 return K
102
103
104 def simulate(
105     t: np.ndarray, s_ref: np.ndarray, u_ref: np.ndarray, s0: np.ndarray, K: np.ndarray
106 ) -> tuple[np.ndarray, np.ndarray]:
107     """Simulate the cartpole
108
109     Args:
110         t (np.ndarray): Evaluation times, shape (num_timesteps,)
111         s_ref (np.ndarray): Reference state s_bar, evaluated at each time t. Shape (num_timesteps, n)
112         u_ref (np.ndarray): Reference control u_bar, shape (m,)
113         s0 (np.ndarray): Initial state, shape (n,)
114         K (np.ndarray): Feedback gain matrix (Ricatti recursion result), shape (m, n)
115
116     Returns:
117         tuple[np.ndarray, np.ndarray]: Tuple of:
118             np.ndarray: The state history, shape (num_timesteps, n)
119             np.ndarray: The control history, shape (num_timesteps, m)
120     """
121
122     def cartpole_wrapper(s, t, u):
123         """Helper function to get cartpole() into a form preferred by odeint, which expects t as the second arg"""
124         return cartpole(s, u)
125
126     # PART (c) #####
127     N = t.size - 1
128     s = np.zeros((t.size, n))
129     u = np.zeros((t.size, m))
130     s[0] = s0
131     for k in range(N):
132         u[k] = K @ (s[k] - s_ref[k]) + u_ref
133         s[k + 1] = odeint(cartpole_wrapper, s[k], t[k : k + 2], (u[k],))[1]
134     u[N] = K @ (s[N] - s_ref[N]) + u_ref
135     # END PART (c) #####
136     return s, u
137
138
139 def compute_lti_matrices() -> tuple[np.ndarray, np.ndarray]:
140     """Compute the linearized dynamics matrices A and B of the LTI system
141
142     Returns:
143         tuple[np.ndarray, np.ndarray]: Tuple of:
144             np.ndarray: The A (dynamics) matrix, shape (n, n)
145             np.ndarray: The B (controls) matrix, shape (n, m)
146     """
147     # PART (a) #####

```

```

148     A = np.eye(n) + dt * np.array([
149         [0, 0, 1, 0],
150         [0, 0, 0, 1],
151         [0, mp * g / mc, 0, 0],
152         [0, (mc + mp) * g / (mc * L), 0, 0],
153     ])
154     B = dt * np.array([[0], [0], [1 / mc], [1 / (mc * L)]])
155     # END PART (a) #####
156     return A, B
157
158
159 def plot_state_and_control_history(
160     s: np.ndarray, u: np.ndarray, t: np.ndarray, s_ref: np.ndarray, name: str
161 ) -> None:
162     """Helper function for cartpole visualization
163
164     Args:
165         s (np.ndarray): State history, shape (num_timesteps, n)
166         u (np.ndarray): Control history, shape (num_timesteps, m)
167         t (np.ndarray): Times, shape (num_timesteps,)
168         s_ref (np.ndarray): Reference state s_bar, evaluated at each time t. Shape (num_timesteps, n)
169         name (str): Filename prefix for saving figures
170     """
171     fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
172     plt.subplots_adjust(wspace=0.35)
173     labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
174     labels_u = (r"$u(t)$",)
175     for i in range(n):
176         axes[i].plot(t, s[:, i])
177         axes[i].plot(t, s_ref[:, i], "--")
178         axes[i].set_xlabel(r"$t$")
179         axes[i].set_ylabel(labels_s[i])
180     for i in range(m):
181         axes[n + i].plot(t, u[:, i])
182         axes[n + i].set_xlabel(r"$t$")
183         axes[n + i].set_ylabel(labels_u[i])
184     plt.savefig(f"{name}.png", bbox_inches="tight")
185     plt.show()
186
187     if animate:
188         fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
189         ani.save(f"{name}.mp4", writer="ffmpeg")
190         plt.show()
191
192
193 def main():
194     # Part A
195     A, B = compute_lti_matrices()
196
197     # Part B
198     Q = np.eye(n) # state cost matrix
199     R = np.eye(m) # control cost matrix
200     K = ricatti_recursion(A, B, Q, R)
201
202     # Part C
203     t = np.arange(0.0, 30.0, 1 / 10)
204     s_ref = np.array([0.0, np.pi, 0.0, 0.0]) * np.ones((t.size, 1))
205     u_ref = np.array([0.0])
206     s0 = np.array([0.0, 3 * np.pi / 4, 0.0, 0.0])
207     s, u = simulate(t, s_ref, u_ref, s0, K)
208     plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance")
209
210     # Part D
211     # Note: t, u_ref unchanged from part c
212     s_ref = np.array([reference(ti) for ti in t])
213     s0 = np.array([0.0, np.pi, 0.0, 0.0])
214     s, u = simulate(t, s_ref, u_ref, s0, K)
215     plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance_tv")
216
217
218 if __name__ == "__main__":
219     main()

```